



Tema 2.

Preparación, limpieza y transformación de datos a escala

Datos textuales, Pandas y trazabilidad de decisiones

Material teórico | Datos Masivos | Universidad Pontificia Comillas

Dr.^a Blanca Hernández Pardo

Índice de contenidos

CONTEXTO PROFESIONAL	1
QUÉ VAS A APRENDER EN ESTA UNIDAD	1
MAPA CONCEPTUAL DE LA UNIDAD	1
1. MARCO TEÓRICO	2
1.1. Preparar datos no es un trámite	2
1.2. Archivo bruto, dataframe de trabajo y archivo preparado	2
1.3. Datos tabulares y datos ordenados	3
1.4. Importación de datos	3
1.5. Inspección inicial	4
1.6. Selección de variables	4
1.7. Estructuración de tablas desde datos anidados	5
1.8. Detección de inconsistencias	5
1.9. Transformaciones descriptivas	6
1.10. Exportación y documentación	6
1.11. Buenas prácticas de preparación	6
2. APLICACIÓN PRÁCTICA	7
3. QUÉ QUEDA FUERA DE ESTA UNIDAD, Y POR QUÉ	7
BIBLIOGRAFÍA	8

Contexto profesional

Imagina que acabas de incorporarte como lingüista computacional junior a **LexiData Observatorio**, una consultora ficticia especializada en análisis documental, tecnologías del lenguaje y apoyo a equipos de comunicación institucional. LexiData no vende respuestas automáticas ni interpreta datos sin revisar: su trabajo consiste en transformar colecciones heterogéneas de textos en materiales ordenados, trazables y preparados para que después puedan analizarse con rigor.

En esta unidad, LexiData recibe un encargo del **Servicio de Observación Documental y Comunicación Pública de una universidad**. El cliente ha recopilado un pequeño volcado de mensajes procedentes de redes sociales en formato JSON. El archivo contiene textos, fechas, identificadores, metadatos de usuario, idioma declarado, fuente de publicación y otros campos técnicos. Antes de elaborar visualizaciones, informes o análisis lingüísticos, el cliente necesita una tabla limpia que permita saber qué contiene la colección y qué decisiones de preparación se han aplicado.

Tu papel en esta simulación es preparar el *dataset* (conjunto de datos). No vas a interpretar la opinión pública, ni vas a clasificar mensajes, ni vas a hacer análisis de sentimiento. Esa prudencia es importante: un análisis fiable empieza mucho antes de las conclusiones. Empieza cuando sabes qué datos tienes, cómo están estructurados, qué información falta, qué campos conviene conservar y qué transformaciones puedes aplicar sin cambiar el significado de los registros.

El producto esperado al final de la unidad será una tabla exportada en CSV con columnas comprensibles, rutas reproducibles y decisiones de limpieza documentadas. El criterio de éxito no será obtener una conclusión llamativa, sino dejar un conjunto de datos preparado para que otra persona pueda revisarlo y continuar el trabajo.

Qué vas a aprender en esta unidad

Al terminar esta unidad deberías ser capaz de:

- distinguir entre archivo bruto, *dataframe* de trabajo y archivo preparado;
- importar datos en Python usando rutas controladas;
- convertir una colección de registros anidados en una tabla;
- seleccionar variables relevantes para un encargo documental;
- revisar valores ausentes, duplicados e inconsistencias básicas;
- crear columnas derivadas descriptivas sin interpretar el contenido textual;
- exportar un *dataset* preparado y documentar las decisiones tomadas.

Mapa conceptual de la unidad

```
Archivo bruto
-> importación reproducible
-> inspección de estructura
-> selección de variables
```

- > tabla de trabajo
- > revisión de inconsistencias
- > transformación descriptiva
- > exportación
- > documentación de decisiones

La idea central es sencilla: los datos no se analizan directamente tal como llegan. Primero se preparan. En proyectos profesionales, esta fase recibe a veces menos atención que los resultados finales, pero condiciona todo lo que se puede afirmar después.

1. Marco teórico

1.1. Preparar datos no es un trámite

La preparación de datos es una fase metodológica, no solo técnica. En un proyecto de análisis lingüístico o documental, cada decisión tomada antes de interpretar los datos puede afectar a las conclusiones posteriores. Si se eliminan registros sin justificarlo, si se cambian nombres de columnas de manera confusa o si se mezclan fases de limpieza e interpretación, el análisis pierde trazabilidad.

La literatura sobre análisis de datos ha insistido en que el trabajo real de los analistas incluye actividades de búsqueda, limpieza, transformación, validación y comunicación, no únicamente la producción de resultados finales (Kandel et al., 2012). En contextos de ciencia de datos con Python, Pandas se ha consolidado precisamente porque ofrece estructuras y operaciones orientadas al trabajo con datos tabulares y etiquetados (McKinney, 2010; The pandas development team, 2020).

Kandel et al. (2012) documentan, a partir de entrevistas a analistas en entornos empresariales reales, que gran parte del tiempo de un proyecto de análisis se dedica a localizar, limpiar y estructurar los datos antes de que pueda empezar cualquier análisis propiamente dicho, muy por delante del tiempo dedicado a producir resultados o visualizaciones finales. Esta observación es la que da sentido a que Datos Masivos dedique una unidad completa a la preparación antes de llegar a la exploración o la visualización.

En esta unidad trabajaremos con un archivo JSON de mensajes. Aunque el *dataset* no es enorme, sí permite simular una situación frecuente: recibir datos textuales semiestructurados y convertirlos en una tabla. La escala no se entiende solo como cantidad de filas; también incluye variedad de campos, presencia de metadatos, formatos anidados y necesidad de reproducibilidad.

1.2. Archivo bruto, *dataframe* de trabajo y archivo preparado

Conviene separar tres objetos:

Archivo bruto. Es la fuente original. Debe conservarse sin modificaciones. Si se altera el archivo bruto, se pierde la posibilidad de volver al punto de partida y comprobar qué decisiones se aplicaron.

Dataframe (tabla de datos) de trabajo. Es la representación tabular que se manipula en Python. En Pandas, un *dataframe* permite organizar datos en filas y columnas, consultar su estructura, seleccionar variables y aplicar transformaciones.

Archivo preparado. Es la versión exportada después de aplicar decisiones documentadas. Puede ser un CSV, un Excel u otro formato adecuado. No sustituye al archivo bruto: lo acompaña como producto derivado.

Esta distinción encaja con la idea de reproducibilidad que se asocia al uso de notebooks: combinar código, explicación y resultados facilita que otra persona pueda entender el proceso seguido (Kluyver et al., 2016). Por eso, en los cuadernos de esta unidad no basta con que el código funcione; también debe explicar qué hace.

1.3. Datos tabulares y datos ordenados

Una tabla útil no es simplemente una tabla que se abre en una hoja de cálculo. Debe tener una organización coherente. Wickham (2014) formuló el principio de *tidy data*: cada variable debe formar una columna, cada observación debe formar una fila y cada tipo de unidad observacional debe estar en una tabla. Esta formulación es muy útil para introducir una disciplina básica de organización.

En el caso de esta unidad, cada fila representa un mensaje. Las columnas representan variables como identificador, fecha, texto, idioma, usuario, ubicación, fuente de publicación o indicadores derivados. Si mezcláramos en una misma celda datos de usuario, texto, fecha y metadatos, la tabla sería difícil de revisar y de reutilizar.

Ejemplo aplicado:

Tabla 1. Correspondencia entre fila, variable y variable derivada según el principio de datos ordenados (elaboración propia).

Fila	Qué representa	Ejemplo de variable
Una observación	Un mensaje individual	`id_str`
Una variable	Una propiedad del mensaje	`texto`
Una variable derivada	Una medida calculada sin interpretar	`longitud_texto`



Figura 1. Del archivo bruto al archivo preparado, pasando por el dataframe de trabajo (elaboración propia).

Esta organización ayuda a preparar el terreno para unidades posteriores: exploración, visualización o minería textual. Pero en esta unidad todavía no analizamos el significado de los mensajes.

1.4. Importación de datos

Importar datos significa cargar un archivo externo en el entorno de trabajo. En los materiales fuente de Data Science con Python se trabaja con `read_csv`, importación de ficheros y creación de *dataframes*. En esta unidad usamos además el módulo estándar `json`, porque el material de trabajo es un archivo JSON.

En un cuaderno reproducible, la ruta del archivo debe estar clara. Si el archivo está en la misma carpeta que el cuaderno, puede usarse una ruta relativa como:

```
ruta_json = Path("U2_tweets_brutos.json")
```

Si el archivo está en otra ubicación, debe indicarse la ruta exacta en el equipo local. En ese caso, el cuaderno debe avisar al estudiante de que modifique la ruta. Lo importante es evitar rutas absolutas ocultas que solo funcionen en el ordenador de quien creó el material.

Una importación responsable incluye una primera comprobación:

```
len(tweets)
type(tweets)
type(tweets[0])
```

Estas instrucciones permiten saber cuántos registros se han cargado y qué tipo de estructura se está manipulando.

1.5. Inspección inicial

Antes de limpiar, hay que mirar. La inspección inicial permite responder preguntas básicas:

- ¿cuántos registros hay?
- ¿qué campos aparecen?
- ¿hay datos anidados?
- ¿cuáles son relevantes para el encargo?
- ¿qué campos parecen técnicos y cuáles pueden interpretarse como variables de análisis?

En un JSON de redes sociales, un registro puede incluir campos principales y subcampos. Por ejemplo, la información del usuario puede estar dentro de `user`; las menciones y URL, dentro de `entities`; y el idioma, dentro de `metadata`. Convertir esa estructura en tabla exige seleccionar y extraer.

No todo campo disponible debe pasar al *dataset* preparado. En un encargo profesional, la selección se justifica por el objetivo. Si el cliente necesita una tabla para revisión documental inicial, quizá basten identificador, fecha, texto, idioma, usuario, ubicación declarada, fuente e indicadores básicos.

1.6. Selección de variables

Seleccionar variables es decidir qué columnas se conservan. Los materiales fuente sobre selección de filas y columnas en *dataframes* permiten trabajar esta operación con Pandas. En esta unidad, la selección debe responder al encargo de LexiData Observatorio.

Una selección mínima puede incluir:

- `id_str`: identificador del mensaje;
- `fecha_publicacion`: fecha del registro;
- `texto`: contenido textual conservado;
- `idioma`: idioma declarado en los metadatos;
- `usuario`: nombre de usuario o identificador público;
- `ubicacion_usuario`: ubicación declarada, si existe;
- `fuentes`: aplicación o cliente desde el que se publicó;
- `num_menciones`, `num_urls`, `num_hashtags`: indicadores descriptivos;
- `es_retuit`: marca básica de difusión;
- `longitud_texto`: longitud del mensaje.

La clave está en que estas variables no interpretan el contenido. Contar URL o medir longitud no equivale a afirmar qué piensa una persona ni qué sentimiento expresa un mensaje.

1.7. Estructuración de tablas desde datos anidados

Los datos anidados son habituales en JSON. Un campo puede contener otro diccionario o una lista. Para llevarlos a una tabla, necesitamos extraer valores concretos.

Ejemplo simplificado:

```
usuario = tweet.get("user") or {}  
ubicacion = usuario.get("location") or None
```

Aquí se accede a la información del usuario. Si el campo no existe o está vacío, se conserva como ausencia. Después podremos decidir si se deja como valor nulo o se sustituye por una etiqueta explícita.

Otro ejemplo:

```
entidades = tweet.get("entities") or {}  
num_urls = len(entidades.get("urls") or [])
```

La operación no lee el significado de las URL. Solo cuenta cuántas aparecen. Es una transformación descriptiva.

1.8. Detección de inconsistencias

Rahm y Do (2000) explican que la limpieza de datos se ocupa de detectar y corregir errores e inconsistencias para mejorar la calidad del conjunto de datos. En esta unidad no automatizamos una limpieza compleja, pero sí aplicamos comprobaciones básicas:

- duplicados por identificador;
- valores ausentes;
- campos vacíos representados como cadena vacía;
- campos técnicos con marcas HTML;
- tipos de datos inesperados;
- registros con idioma distinto al esperado.

Detectar no significa borrar. Si 45 mensajes no tienen ubicación declarada, eliminarlos sin más sería una decisión fuerte. Podría sesgar el conjunto si los mensajes sin ubicación comparten alguna característica. Una decisión más prudente consiste en etiquetar la ausencia como `sin_ubicacion` y documentarlo.

1.9. Transformaciones descriptivas

Transformar datos significa crear una versión más útil para el análisis. Pero no toda transformación es aceptable en cualquier fase. En esta unidad solo se aplican transformaciones descriptivas:

- limpiar etiquetas HTML de la columna fuente;
- normalizar espacios en el texto;
- contar menciones, hashtags y URL;
- calcular longitud del texto;
- marcar si el mensaje empieza como *retuit*;

- reemplazar ubicaciones vacías por `sin_ubicacion`.

Estas decisiones preparan el *dataset* sin introducir conclusiones. No se infiere ideología, emoción, tema ni calidad textual. Ese límite protege la coherencia metodológica de la unidad.

1.10. Exportación y documentación

Exportar el *dataset* preparado crea un producto que puede revisarse fuera del cuaderno. En Pandas, la operación habitual es:

```
df_limpio.to_csv("U2_tweets_preparados_profesor.csv", index=False,
encoding="utf-8")
```

El parámetro `index=False` evita añadir una columna de índice artificial. La codificación `utf-8` ayuda a conservar caracteres propios del español. Después de exportar, conviene abrir el archivo o volver a cargarlo para comprobar que tiene las columnas esperadas.

La documentación es parte del producto. Un CSV sin explicación puede ser ambiguo. Debe indicarse qué se ha hecho, qué se ha conservado, qué se ha cambiado y qué queda pendiente de validación.

1.11. Buenas prácticas de preparación

Para esta unidad, consideraremos buenas prácticas las siguientes:

1. Conservar el archivo bruto sin modificar.
2. Trabajar con una copia local dentro de la unidad.
3. Usar rutas claras y modificables.
4. Separar inspección, limpieza, transformación y exportación.
5. Nombrar columnas de forma comprensible.
6. No eliminar registros sin justificación.
7. No mezclar preparación con interpretación.
8. Documentar decisiones en el cuaderno.
9. Exportar una versión final revisable.
10. Dejar constancia de límites y pendientes.

Estas reglas son sencillas, pero constituyen una base sólida para trabajar después con corpus, minería textual o visualización.

2. Aplicación práctica

En el cuaderno de clase, prepararás el archivo `U2_tweets_brutos.json`. El flujo será:

1. cargar el JSON;
2. inspeccionar su estructura;
3. extraer campos útiles;

4. convertirlos en *dataframe*;
5. revisar ausentes, idioma y duplicados;
6. crear columnas derivadas descriptivas;
7. exportar el CSV;
8. justificar decisiones.

El resultado no será una conclusión sobre los mensajes. Será una tabla preparada para que el equipo de LexiData pueda decidir, en una fase posterior, qué análisis es metodológicamente adecuado.

3. Qué queda fuera de esta unidad, y por qué

Por qué no se hace análisis de sentimiento ni se clasifica el contenido en temas. Ambas tareas pertenecen a Programación orientada a PLN, asignatura que ya cursaste. Concretamente, la unidad sobre clasificación temática de textos enseña a representar un texto como un vector de palabras (o de TF-IDF) y a compararlo, mediante similitud del coseno, con un vector prototipo de cada categoría, asignando la etiqueta con mayor puntuación; sus prácticas aplican además esa misma lógica, con un clasificador entrenado sobre ejemplos etiquetados, a un caso de valorar mensajes como positivos o negativos — un caso particular de lo que aquí llamamos análisis de sentimiento. A partir de la tabla que prepares en esta unidad, si lo desearas, con los conocimientos que ya tienes de esa asignatura podrías construir vectores de palabras a partir de la columna texto, definir uno o varios vectores prototipo por categoría (o entrenar un clasificador con ejemplos etiquetados) y calcular la similitud de cada mensaje con esos vectores para asignarle una categoría o una polaridad, tal y como se narra en la unidad sobre clasificación temática de textos de esa asignatura anterior. No lo hacemos aquí porque el objetivo de esta unidad es dejar los datos preparados para que esa fase, si llega, pueda hacerse bien — no porque esté fuera de tu alcance.

Por qué no se extraen conclusiones sociopolíticas. Incluso la unidad de Programación orientada a PLN que más se acerca a este terreno —la que trabaja con corpus reales de redes sociales para detectar sesgo y lenguaje valorativo— enseña explícitamente a no dar ese salto: su objetivo es producir indicios (marcas de polaridad, patrones de coocurrencia) para que una persona los interprete después, no cerrar un veredicto sobre opinión pública o intención. La misma cautela metodológica que se enseña allí se aplica aquí, con más razón todavía, porque en esta unidad ni siquiera has llegado a la fase de análisis: estás preparando los datos.

Por qué no se eliminan registros solo porque resulten incómodos o incompletos. No es una cuestión de estilo, sino de honestidad metodológica. Descartar datos por razones distintas a un criterio de calidad documentado (duplicados, campos corruptos) equivale a decidir en la práctica qué parte de la realidad se muestra, sin decirlo. Kandel et al. (2012) señalan justo este tipo de decisión como una de las más delicadas del trabajo real con datos, y por eso debe quedar siempre documentada, nunca oculta.

Por qué no se inventan metadatos que no aparecen en la fuente. Esta norma se aplica a toda la asignatura: cualquier columna, valor o campo que uses debe poder rastrearse hasta el archivo bruto original. Inventar un metadato ausente, por inofensivo que parezca, rompe la trazabilidad que esta unidad entera existe para enseñar.

Bibliografía

Kandel, S., Paepcke, A., Hellerstein, J. M. y Heer, J. (2012). Enterprise data analysis and visualization: An interview study. *IEEE Transactions on Visualization and Computer Graphics*, 18(12), 2917-2926. <https://doi.org/10.1109/TVCG.2012.219>

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B. E., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S. y Willing, C. (2016). Jupyter Notebooks - a publishing format for reproducible computational workflows. In F. Loizides & B. Schmidt (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87-90). IOS Press. <https://doi.org/10.3233/978-1-61499-649-1-87>

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>

Rahm, E. y Do, H. H. (2000). Data cleaning: Problems and current approaches. *IEEE Data Engineering Bulletin*, 23(4), 3-13.

The pandas development team. (2020). *pandas-dev/pandas: Pandas* [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.3509134>

Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1-23. <https://doi.org/10.18637/jss.v059.i10>